Explainable Autonomic Cybersecurity For Industrial Control Systems

Valeti Manoj
Johns Hopkins University
mvaleti1@jhu.edu

Shao Wenda
Johns Hopkins University
wshao12@jhu.edu

Niu Sihan
Johns Hopkins University
sniu7@jhu.edu

Christopher Rouff
Johns Hopkins Applied Physics Laboratory
christopher.rouff@jhuapl.edu

Lanier Watkins
Johns Hopkins University
lanier.watkins@jhuapl.edu

Aviel Rubin
Johns Hopkins University
rubin@jhu.edu

2023 IEEE 13th Annual Computing and Communication Workshop and Conference (CCWC) | 979-8-3503-3286-5/23/$31.00 ©2023 IEEE | DOI: 10.1109/CCWC57344.2023.10099307

***Abstract*—Industrial Control System (ICS) networks are the most targeted systems by attackers in the world, because they are the life-blood of any country's Critical Infrastructure. In this work-in-progress paper, we propose a novel machine learning-based (decision trees) intrusion detection system for ICS networks inspired by the principals of autonomic cybersecurity with the addition of explainability. Since autonomic systems are autonomous and can be preventative (i.e., active) by nature, the added benefit of explainability helps to better establish human trust in an era where human confidence in artificial intelligence (AI) is low. In the spirit of full transparency, we clearly explain our methodology, data collection, feature development, model training and testing, and results. Our resulting models achieved an accuracy of over 95%, and we also illustrate how information can be extracted from the nodes in the decision trees to clearly explain how our models arrived at their decisions.**

***Index Terms*—Intrusion Detection, Explainable, Machine Learning, Industrial Control Network**

I. Introduction

Attacks on industrial control systems (ICS) have increased dramatically in both type and quantity over the last ten years. One-third of US industrial control systems were maliciously attacked in the first half of 2021 [1]. Industrial control systems are inextricably linked to the well being of society and the economy. These attacks can result in significant financial losses, and stolen data can endanger society and human security. These attacks are being increasingly carried out not only because of the convergence of operational technology (OT) and information technology (IT), but also because criminals have discovered that they can profit from attacking ICS.

According to recent research, there was a 41% increase in ICS vulnerabilities between February and August 2022 [1], continuing the trend of rising attacks. These are the known vulnerabilities that have been collected; there are many more that have not yet been identified. Patching these vulnerabilities is also problematic. More than one-fifth of the vulnerabilities are hard-to-fix zero-day flaws, and more than one-third of the vulnerabilities lack a patch. The US government is well aware of the criticality of ICS. According to a report from the US National Security Agency, hackers can scan, destruct, and take control of some devices using OT network permissions they obtained using zero-day vulnerabilities. A large amount of damage could be done to our Critical Infrastructure if hackers gained access to electrical, gas-related or other industrial machinery [2].

Our contributions in this work-in-progress paper includes: (1) the transparent implementation of a 2-Level Self-Aware Reasoning System (SARS) [3] using decision trees and (2) a discussion on how to make this autonomic system explainable. The rest of the paper includes the following sections. In Section 2 we discuss related work, and in Section 3 we describe our Methodology. Next in Section 4 we discuss our threat model, and in Section 5 we talk through our data science procedure. Then in section 6 we mention our experimental evaluation, and in section 7 we reveal our results. Finally in section 8 we discuss our results, and in section 9 we conclude with a summary and future work discussion.

II. Related Works

IoT technologies are becoming more common, but globally cybersecurity protection measures for them are insufficient. As described in [6], systematic autonomous cybersecurity knowledge can be used to solve the issues faced by ICS. Hackers and those that defend their networks against hackers essentially are involved in an arms race, and autonomic computing systems, which can be self protecting, help with this [7].

The authors in [8] is consistent with our approach, they use Supervisory Control and Data Acquisition (SCADA) network data to identify attacks. They conducted an experiment with 1,500 packets to extract some of their characteristics, such as the size and transmission rate of the packets. The results were very promising, and we built on their advances by collecting larger datasets and collecting more relevant data to consider the impact of attacks.

There is also a study identifying SCADA on ICS networks, which conducted their research on recurrent networks, and they used a monitoring tool called *Bro* [9], [10]. The tool *Bro* was used to help clean our data to obtain the data we needed. The previous study only conducted theoretical research and did not really attack the network. In our research, we set up an ICS network, developed a threat model, executed attacks, collected

979-8-3503-3286-5/23/$31.00 ©2023 IEEE

Authorized licensed use limited to: University of Maryland Baltimore Cty. Downloaded on November 02,2026 at 20:38:41 UTC from IEEE Xplore. Restrictions apply.

data, and analyzed the results of our malicious detection system.

For security concerns, effective intrusion detection methods are crucial. A host-based intrusion detection system was built using decision trees in a recent study [11]. A C4.5 detector built on the Consolidated Tree Construction (CTC) algorithm is also used by the system. In addition to offering a random sampling algorithm to handle the supervised data, it also creates a part that selects and filters data based on various features. This has significant ramifications for both the logs we plan to gather and the keyboard input data. The accuracy rate of the outcome is up to 99.95% [11].

III. Methodology

In our approach, a virtual ICS network containing a SCADA device and an engineering workstation was set up. The SCADA device continuously took in randomly generated process data and performed arbitrary calculations that were accessible from the engineering workstation. System vitals were collected from the SCADA device, while the user behavior was monitored on the engineering workstation. The data was then separated into: (1) SCADA system status, (2) SCADA networking, and (3) engineering workstation user activity.

For each category, a machine learning (ML) model was trained to identify our threat model. This is the crux of our a 2-Level SARS using decision trees. At Level 1, there were multiple anomaly detectors that continuously monitored the behavior of the SCADA device, user, and network. At Level 2, all of the outputs of the detectors were aggregated and a decision was made on the abnormality of the network. Then we discussed how to determine the explainability of the system.

IV. Threat Model

We used a simple threat model (i.e., script kiddie) since our primary goal is only to demonstrate all of the steps in building an autonomic intrusion detection system and illustrate explainability. To produce anomaly data, several attacks were executed and data was collected.

The first attack was port scanning on the network. Although this activity is not really an attack, it is considered malicious intending behavior in the reconnaissance phase of hacking. The port scans were executed three times, each time it took approximately 20 seconds.

The second attack was a denial of service (i.e., Ping of Death) on the SCADA device. The command sent ICMP packets with a payload of 65,500 bytes continuously for two minutes. Another attack method, a SYN flood attack, was used for an hour.

The last attack was a password brute-force attack on the SCADA web interface. To mimic the behavior of a brute-force script, the HTTP request for the web login was captured and replayed with curl without any pause in between. The brute-force script ran for two hours. In total for our threat model, there were $49,244$ abnormal samples created.

V. Data Science Procedure

A. Data Collection

The following paragraphs describes our data collection approach. Python scripts were used to gather data on the SCADA VM and the workstation VM. On the SCADA VM, a library named $psutils$ was used. The collected metrics were: CPU percent, CPU time, memory usage, network bytes rate, network packets rate and network connections count. Since $psutil$ only provides the total sent/received network bytes and the total sent/received packets, the rate was calculated by the script on a 1-second basis. On the engineering workstation, the $pynput$ library was used to record the user input events, including key press and releases, mouse cursor movements, mouse button clicks and mouse wheel scroll.

All data was collected at one sample per second and then sent to the server running on the collection node (i.e., Linux VM). The data was encoded in JSON and sent through UDP. Compared to HTTP, it was found to have a minimal impact on the performance of the SCADA and engineering workstation. The data was then stored in a CSV file.

Two categories of data were collected for training the micro intrusion detectors: (1) data of normal operations and (2) system under attack data. To simulate the normal operations, the RapidSCADA ran dummy processes, such as sine wave and square wave that are similar to randomly probing voltage and switch toggles. The user occasionally opened Firefox on the workstation and visited the SCADA web interface. The user logged in and checked on all running functions, then viewed the history data. A total of $58,477$ normal samples were collected.

B. Feature Engineering

We collected two types of feature data: (i) data corresponding to the SCADA device and (ii) data corresponding to user behavior. These two data sets were not mapped within the same time frequency. So to make our datasets have the same time frequency between each data record we sampled the user behavior data in a way that matched the frequency of the collected SCADA data. Once this was done we performed data cleaning using various data mining techniques to find the correlation between each of the attributes present in the datasets. We then removed all those features from the datasets that remained constant, even when an attack is performed on the ICS network. Excluding these features from the datasets helped the model to better correlate the features that change when an attack was happening. A correlation heatmap was plotted to visualize the contribution of each attribute to the label, as shown in Fig 1 and Fig 2. Data processing requires normalization of the cleaned data. Apart from the heatmap we also used WEKA's data visualization tool to understand the distribution of data points with changes in time. Later data was cleaned and normalized to minimize the effect of outliers on the model.

Authorized licensed use limited to: University of Maryland Baltimore Cty. Downloaded on November 02,2026 at 20:38:41 UTC from IEEE Xplore. Restrictions apply.

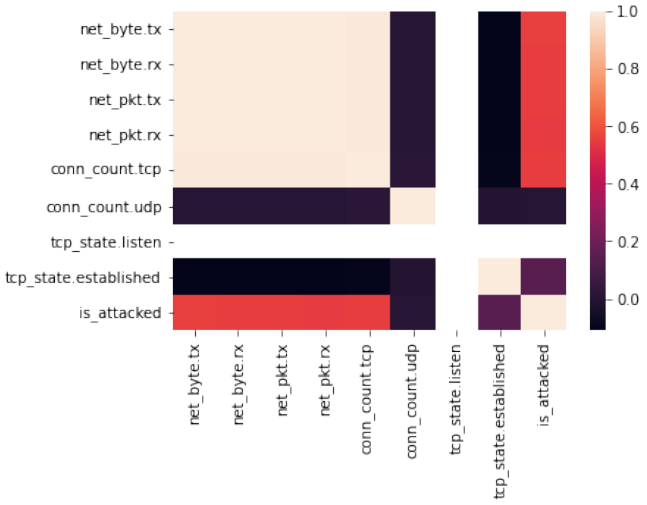


Fig. 1. Network Data Correlation Heatmap

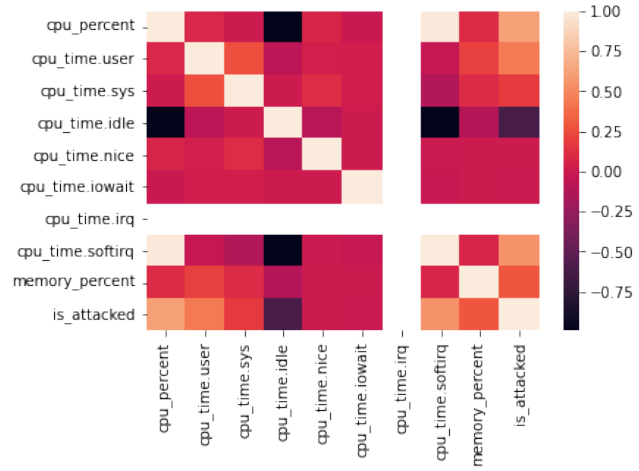


Fig. 2. System Data Correlation Heatmap

C. Model Training & Testing

Once the data was pre-processed, it was transferred to the machine learning models for training. In our technique of anticipating the attack, we employed three models to train three separate ICS network behaviors, namely *user-behavior*, *CPU performance* and *Network traffic*. We refer to these as explainable micro intrusion detectors. We set aside 20% of the total data gathered as test data; this data is never used in the training phase of any models. We trained three models multiple times using the remaining 80% of the data to avoid overfitting and underfitting.

After training the models (i.e., explainable micro intrusion detectors), we utilized test data to predict the results of each model. These results were then forwarded to an aggregator, which makes a decision based on a voting mechanism. If more than one model predicts an attack, the voting system returns "1" (ICS under attack); otherwise, it will return "0" (ICS is not under attack). The projected output was then compared to the actual values. This procedure was performed using various classification models, such as *Decision Tree*, *Random Forest*, *K-Nearest Neighbors*, *Support Vector Classifier*, and *Gaussian Naive Bayes Classifier*. The results of these models are shown in Table 1.

TABLE I
MODELS' AND THEIR ACCURACY

Model Name	*Accuracy*
Decision Tree	95.372%
Random Forest	95.316%
KNN	95.061%
SVC	64.692%
Gaussian Naive Bayes	64.673%

TABLE II
DECISION TREE METRICS

Class	*Precision*	*Recall*	*F1-score*
0	1.0	0.92	0.96
1	0.9	1.0	0.95

TABLE III
RANDOM FOREST METRICS

Class	*Precision*	*Recall*	*F1-score*
0	1.0	0.92	0.96
1	0.9	1.0	0.95

TABLE IV
GAUSSIAN NAIVE BAYES METRICS

Class	*Precision*	*Recall*	*F1-score*
0	1.0	0.61	0.76
1	0.22	1.0	0.36

TABLE V
SUPPORT VECTOR CLASSIFIER METRICS

Class	*Precision*	*Recall*	*F1-score*
0	1.0	0.61	0.76
1	0.22	1.0	0.36

TABLE VI
K-NEAREST NEIGHBORS METRICS

Class	*Precision*	*Recall*	*F1-score*
0	1.0	0.92	0.96
1	0.89	1.0	0.94

VI. EXPERIMENTAL EVALUATION

A. Experimental Setup

The experimental environment was set up on 3 different VMWare Workstations (17.0), see Figure 3. The host machine was a laptop with intel Core i7-12700H (12 core) and 32GB 4800 MHz memory. The SCADA VM was assigned one virtual CPU and 1 GB of RAM to simulate a hardware device

Authorized licensed use limited to: University of Maryland Baltimore Cty. Downloaded on November 02,2026 at 20:38:41 UTC from IEEE Xplore. Restrictions apply.

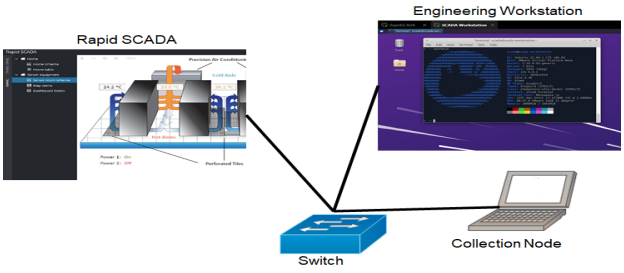


Fig. 3. Experimental Setup

with a limited hardware specification. The OS of this VM was Ubuntu server 22.04 minimal and RapidSCADA 6.0.0 was installed on top of the Ubuntu server. RapidSCADA provides a web interface for users to monitor running status, view historical data, and toggle features on and off.

The engineering workstation VM was assigned 4 virtual CPUs and 2 GB of RAM and Kubuntu (Ubuntu with KDE as desktop environment) was installed. It had the Firefox web browser so users could access the SCADA web interface.

Another Linux VM (i.e., collection node) collected and stored data from the SCADA device and workstation. All three VMs were connected to the same virtual network. To simulate a realistic environment similar to a physical setup, the virtual network was set with a 3 milliseconds latency with VMWare's Adapter Advanced Settings.

B. Experimental Procedure

Our experimental procedure was relatively simple, because our goal was to design and setup a fully functioning autonomic cybersecurity system such that its resulting actions were explainable. We took the following steps: (1) we setup a SCADA network, (2) collected data across several domains (i.e., SCADA device, user, and SCADA network) of the SCADA network, (3) developed and executed a threat model, (4) developed a 2-level autonomic cybersecurity system using decision trees, (5) extracted features from the data, (6) trained and tested machine learning classifiers (including decision trees) to identify the threat model and aggregate micro intrusion detector data, and (7) used the nodes of the decision trees to explain the actions of the autonomic system.

VII. Results

We utilized five classification models and compared their results to determine the best classification model that suited our ICS network. We discovered that the "Decision tree" model had the highest accuracy of 95.367% with F1-scores of 0.96 (class 0) and 0.95 (class 1), while Gaussian Naive Bayes had the lowest accuracy of 64.6% with F1-scores of 0.76 (class 0) and 0.36 (class 1). The random forest classifier had the second highest accuracy of 95.31% with F1-scores of 0.96 (class 0) and 0.95 (class 1). K-Nearest Neighbors(with nearest neighbors=2) had an accuracy of 95.05% with F1-scores of 0.96(class 0) and 0.94 (class 1), and the Support Vector Classifier (SVC) had an accuracy of 64.69% with F1-scores 0.76 (class 0) and 0.36 (class 1). The poor accuracy and F1 scores for the models like SVC and Gaussian NB suggest that these models are not effective in detecting attacks on an ICS network. Table I to Table VI depicts the results when the same training model is used for all three IDS.

Table VI to Table XII shows results when we used different combinations of models when training. The results produced by a combination of different models are quite similar to the prior case where a case model is used to train all three different behaviors.

TABLE VII
K-Nearest Neighbors Metrics

Userbehaviour Model	*CPU Model*	*Network Model*	*Accuracy*
KNN	RandomForest	DecisionTree	95.2146%
KNN	DecisionTree	RandomForest	95.2053%
RandomForest	KNN	Decision Tree	95.307%
RandomForest	DecisionTree	KNN	95.316%
Decision Tree	KNN	RandomForest	95.358%
Decision Tree	RandomForest	KNN	95.381%

TABLE VIII
KNN & RandomForest & DecisionTree

Class	*Precision*	*Recall*	*F1-score*
0	1.0	0.92	0.96
1	0.9	1.0	0.94

TABLE IX
KNN & DecisionTree & RandomForest

Class	*Precision*	*Recall*	*F1-score*
0	1.0	0.92	0.96
1	0.9	1.0	0.94

TABLE X
RandomForest & KNN & Decision Tree

Class	*Precision*	*Recall*	*F1-score*
0	1.0	0.92	0.96
1	0.9	1.0	0.95

TABLE XI
RandomForest & DecisionTree & KNN

Class	*Precision*	*Recall*	*F1-score*
0	1.0	0.92	0.96
1	0.9	1.0	0.95

TABLE XII
Decision Tree & KNN & RandomForest

Class	*Precision*	*Recall*	*F1-score*
0	1.0	0.92	0.96
1	0.9	1.0	0.95

Authorized licensed use limited to: University of Maryland Baltimore Cty. Downloaded on November 02,2026 at 20:38:41 UTC from IEEE Xplore. Restrictions apply.

TABLE XIII
DECISION TREE & RANDOMFOREST & KNN

Class	*Precision*	*Recall*	*F1-score*
0	1.0	0.92	0.96
1	0.89	1.0	0.95

VIII. DISCUSSION

A. Model Performance

In the above results, it is clear that the Naive Bayes model and the SVC perform significantly worse than the other models by approximately 30%. For the Naive Bayes approach, the assumption is that the explanatory variables in the dataset are independent. While in our dataset, variables such as CPU times and CPU percentage are correlated with each other, which could impact the model performance.

The SVC model, on the other hand, performs best if the data is balanced and scaled properly. In this experiment, the two categories had a similar amount of records in the dataset, therefore no data balancing or normalization is applied, which negatively influences the accuracy of the model.

B. Explainable Models

Compared to the neural network, where the middle layer is considered a BlackBox, some of the models used in our experiments are explainable. Specifically, a decision tree is a tree-like structure that consists of nodes, branches, and leaves, and each node in a decision tree represents a decision or a choice that needs to be made and the branches represent the possible outcomes of that decision. Therefore, by inspecting the model, each step of the decision-making process can be reviewed. However, all three decision tree models trained were complex in structure. They had a depth of over 20 layers and contained thousands of nodes, which is difficult to interpret.

To simplify, the models were trained again with a max depth limit of 5. The figures in the appendix illustrate the visualization of these decision trees. Compared to unlimited depth models, the accuracy only decreases from 95.372% to 95.25%, which is an acceptable trade-off. The model requires less memory to run and uses fewer steps to get to the result, resulting in faster execution.

Simplifying the models also reduces the complexity, which makes interpretation easier, because the decision process is shorter to track. In the performance model (See Figure 4), the pruned decision tree can be transformed into a collection of boolean conditions. These conditions that help to identify the abnormal behaviour are shown in Table 14. Similarly, in the network model (See Figure 5), the boolean conditions for this model appear in Table 15.

In Table 14, the decision condition listed in the first row covers 97% (30,586 / 31,575) of the abnormal instances classified by the performance model. Based on our analysis, this condition, which only consists of memory_percent and cpu_time.sys, tests for long running memory bound processes or activities, and thus we posit that this model's malicious detection capabilities is mostly governed by this condition. This makes sense based on our threat model, which contains denial-of-service and brute force attacks. These attacks can induce long running memory bound processes on a small memory and CPU device (i.e., SCADA device).

A similar approach can be taken with the condition in the sixth row in Table 15. This condition covers 86% (24,296 / 28,299) of the abnormal instances classified by the network model. This condition, which only consists of conn_count.tcp, net_byte.rx, and net_byte.tx, likely tests for multiple connections that send and receive small files. Again, this condition is in sync with the network behavior from our threat model, specifically a brute force attack or multi-threaded denial-of-service attack, and thus this model's abnormal decisions are based on seeing network behavior based on multiple connections that transfer and receive small files.

TABLE XIV
EXPLAINABLE PERFORMANCE MODEL USING THE PRUNED DECISION TREE

Instances	Decision Condition	Behavior
30586	**memory_percent** >64.542 & **cpu_time.sys** <= 69.0	Abnormal
944	memory_percent <= 63.865 & memory_percent >63.286	Abnormal
9	memory_percent >63.865 & memory_percent <= 63.868 & cpu_time.softirq >2.0	Abnormal
1	memory_percent <= 64.542 & memory_percent >63.868 & cpu_percent <= 8.65 & cpu_time.idle <= 82.5	Abnormal
35	memory_percent <= 64.542 & memory_percent >64.502 & cpu_percent >8.65	Abnormal

IX. SUMMARY AND FUTURE WORK

In this work-in-progress paper, we share the details of the implementation of a 2-Level SARS using decision trees, and we demonstrate an analysis approach to explain its decisions. Also, we do recognize that we did not make our system proactively response to threats. This limitation and others we will address in future work.

In future work, we will extend the scope of our explainable micro intrusion detectors to include other attributes of the ICS network. Also, since we did not incorporate a mechanism in our models to provide feedback during the training phase, we will investigate using methods, that continually feed the findings back to the model to avoid underfitting and overfitting, and also help the model predict the output more effectively. In addition to traditional denial of service (DoS) and brute-force attacks, other attacks will be included in our threat model. It may also be beneficial to have our explainable micro intrusion detectors identify the category of the attack, since this may help the system to automatically take appropriate counter-measures to mitigate the threat. Finally, we will investigate different approaches to having our 2-Level SARS proactively respond to threats.

Authorized licensed use limited to: University of Maryland Baltimore Cty. Downloaded on November 02,2026 at 20:38:41 UTC from IEEE Xplore. Restrictions apply.

TABLE XV
EXPLAINABLE NETWORK MODEL USING THE PRUNED DECISION TREE

Instances	Decision Condition	Behavior
97	conn_count.tcp <= 11.5 & net_pkt.rx >72.5	Abnormal
2294	conn_count.tcp >13.5 & net_byte.rx >18289.0	Abnormal
4	conn_count.tcp <= 11.5 & net_pkt.rx <= 72.5 & tcp_state.established >2.5 & net_byte.tx <= 2296.5 & net_byte.rx >1873.0	Abnormal
38	net_byte.rx <= 1839.5 & tcp_state.established <= 4.5 & conn_count.tcp >37.5	Abnormal
1482	conn_count.tcp >11.5 & conn_count.tcp <= 13.5 & net_byte.rx <= 1839.5 & tcp_state.established >4.5 & tcp_state.established <= 5.5	Abnormal
24296	**conn_count.tcp** >11.5 & **conn_count.tcp** <= 13.5 & **net_byte.rx** >1839.5 & **net_byte.tx** <= 17747.5	Abnormal
88	net_byte.rx >1839.5 & net_byte.rx <= 18289.0 & conn_count.tcp >13.5 & conn_count.tcp <= 14.5	Abnormal

REFERENCES

[1] J. Cornelius, "Why Are Industrial Control System Attacks Increasing?" Infosecurity Magazine Online. Available At:https://www.infosecurity magazine.com/opinions/why-industrial-attacks-increasing/ 9/22/2021

[2] G. Cluley, "US Government Warns of New Malware Attacks On ICS/SCADA Systems". Tripwire Online. Available At: 4/14/2022

[3] Bellman, Kirstie, Christopher Landauer, Nikil Dutt, Lukas Esterle, Andreas Herkersdorf, Axel Jantsch, Nima TaheriNejad, Peter R. Lewis, Marco Platzner, and Kalle Tammemäe. "Self-aware cyber-physical systems." ACM Transactions on Cyber-Physical Systems 4, no. 4 (2020), pages 1-26.

[4] Hosic, Jasenko, Jereme Lamps, and Derek H. Hart. "Evolving decision trees to detect anomalies in recurrent ICS networks." 2015 World Congress on Industrial Control Systems Security (WCICSS). IEEE, 2015.

[5] R. Sommer and V. Paxson. Outside the Closed World: On Using Machine Learning For Network Intrusion Detection. In IEEE Symposium on Security and Privacy, SP 2010, pages 305–316, Mar. 2010.

[6] Rouff, Christopher, et al. "SoK: Autonomic Cybersecurity-Securing Future Disruptive Technologies." 2021 IEEE International Conference on Cyber Security and Resilience (CSR). IEEE, 2021.

[7] Sterritt, Roy. "Autonomic computing." Innovations in systems and software engineering 1(1) (2005), pages 79-88.

[8] L. A. Maglaras and J. Jiang. Intrusion Detection in SCADA systems using Machine Learning Techniques. In Science and Information Conference, SAI 2014, pages 626–631, 2014

[9] Kaya, Yigitcan, Sanghyun Hong, and Tudor Dumitras. "Shallow-deep networks: Understanding and mitigating network overthinking." International conference on machine learning. PMLR, 2019.

[10] M. Mantere, M. Sailio, and S. Noponen. A Module for Anomaly Detection in ICS Networks. In Proceedings of the 3rd international conference on High confidence networked systems, HiCoNS '14, pages 49–56, Apr. 2014

[11] Panigraphi, Ranjit, Samarjeet Borah, Akash Kumar Bhoi, Muhammad Fazal Ijaz, Moumita Pramanik, Yogesh Kumar and JRutvij H haveri. A consolidated decision tree-based intrusion detection system for binary and multiclass imbalanced datasets. Mathematics, MDPI, 9(7), pages 751, 2021.

APPENDIX A
DECISION TREE VISUALIZATION

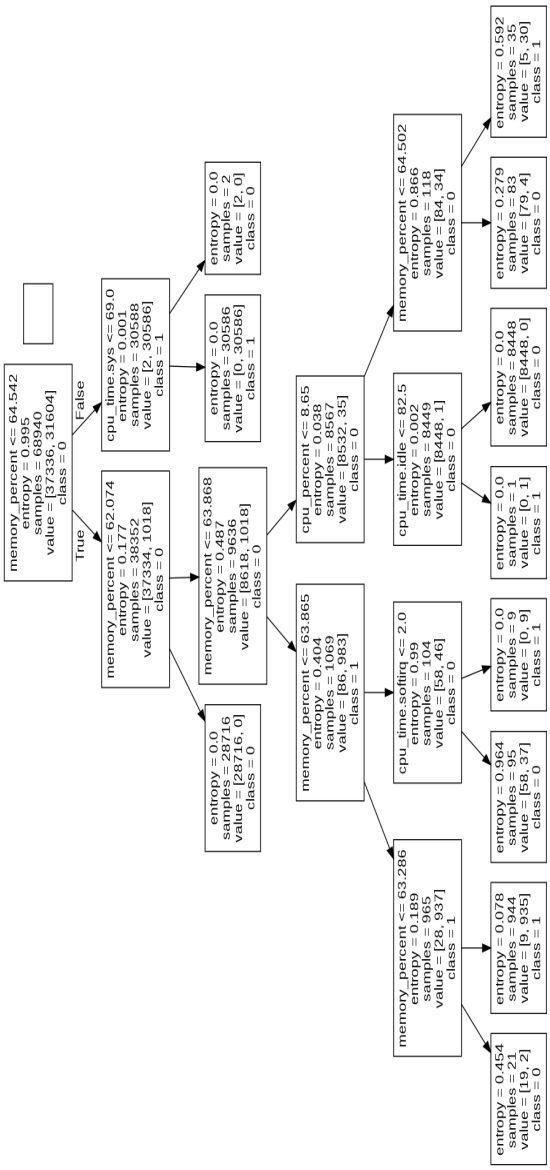


Fig. 4. Decision Tree: Performance Model

Authorized licensed use limited to: University of Maryland Baltimore Cty. Downloaded on November 02,2026 at 20:38:41 UTC from IEEE Xplore. Restrictions apply.

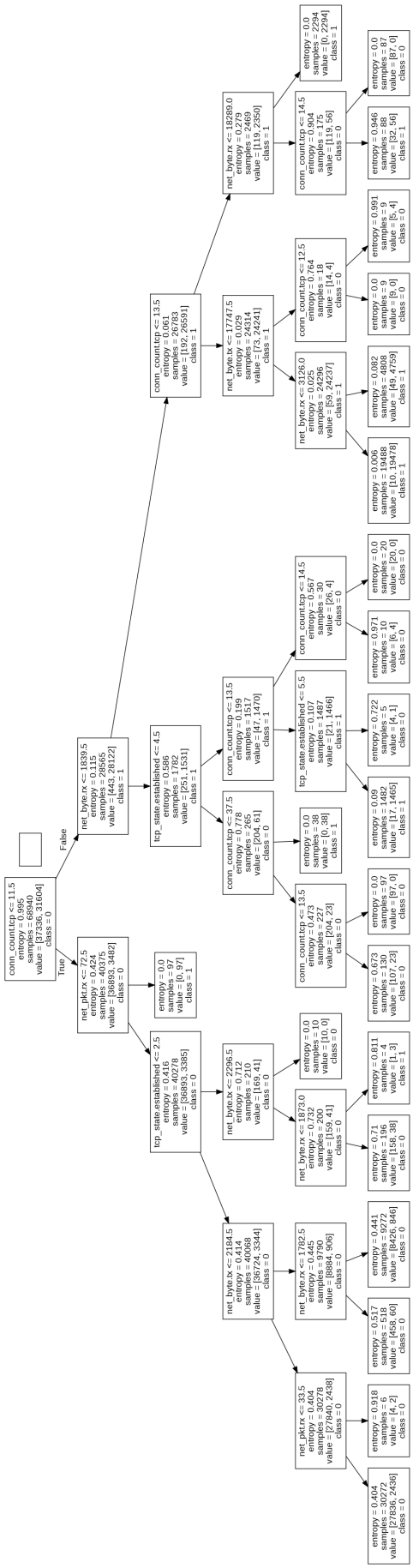


Fig. 5. Decision Tree: Network Model

Authorized licensed use limited to: University of Maryland Baltimore Cty. Downloaded on November 02,2026 at 20:38:41 UTC from IEEE Xplore. Restrictions apply.